

Auditoría de Seguridad RadFor-360 — 2026-09-04

Alcance: BYOK, variables de entorno, aislamiento multi-tenant (RLS) y permisos de base de datos.

Método: inspección directa de código + verificación en vivo contra la base de datos de producción (Supabase, proyecto `ozivmsvxbdtjkzleqbcy` , compartido con el proyecto raíz AntigraVity JS). Ninguna afirmación de este documento carece de evidencia citada (comando ejecutado o archivo:línea).

Resumen ejecutivo

#	Hallazgo	Estado al cierre de hoy
1	BYOK (API key de Gemini) en <code>localStorage</code> sin cifrar	● Descartado — ya remediado antes de esta auditoría (commit <code>173a07e</code> , 2026-08-24)
2	Variables de entorno desalineadas (<code>.env.example</code> vs código real)	● Corregido — documentación y <code>render.yaml</code> actualizados
3	Rol de conexión principal con <code>BYPASSRLS=true</code> — RLS nunca protegió nada	● Mitigado parcialmente — rol nuevo aditivo creado y en uso para 7 archivos; ~400 rutas del pool principal siguen sin backstop de RLS (deuda conocida, ver §5)
4	5 tablas con RLS desactivado y con <code>anon / authenticated</code> (claves públicas) con privilegios completos	● Vector de ataque cerrado hoy (REVOKE aplicado y verificado) — ● RLS + políticas de tenant aún no escritas para estas 5 tablas (ver §4)

Lectura correcta de esta tabla: la fuga de datos más grave y explotable sin autenticación (hallazgo 4) está cerrada. El hallazgo 3 (arquitectura de aislamiento multi-tenant completa) sigue con trabajo real pendiente — no se cierra en una sesión.

1. BYOK en localStorage — descartado

Ver auditoría de 006-devsecops-infraestructura (misma fecha). La llave BYOK vive en `useState` , se envía cifrada (AES-256-GCM) al backend, nunca toca el navegador. Documentado en `docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md` §9 y en `RADFOR360_REPORTE_TECNICO.md` §5.1 (nota `[RESUELTO – 2026-09-04]`).

2. Variables de entorno — corregido

- `.env.example` ya no afirma falsamente que `SUPABASE_SERVICE_KEY` "no se usa" — el código sí la usa activamente (`backend/config/supabase.config.js` , `database.config.js` , `server.js`).
- `render.yaml` completado con las 22 variables que el código usa y no estaban declaradas (Stripe, Wompi, Supabase, Sentry backend, PostHog, Resend, `DATABASE_URL_TENANT_SCOPED`).
- Pendiente de verificación manual (no ejecutable desde este entorno): confirmar en el dashboard de Render cuáles de esas variables ya tienen valor real cargado.

3. `BYPASSRLS=true` en el rol principal

Verificado en vivo: `SELECT current_user, rolbypassrls FROM pg_roles WHERE rolname = current_user → postgres , rolbypassrls=true` . Esto significa que ninguna política RLS ha protegido nunca nada en este proyecto a través del pool principal.

Remediación aplicada: arquitectura de doble pool, aditiva (no se tocó ni revocó nada del rol `postgres` existente, por ser una base de datos compartida con el proyecto raíz):

- Migración `backend/migrations/053_rls_scoped_role.sql` — crea el rol `rf360_rls_scoped` (sin `BYPASSRLS` / `SUPERUSER`), con GRANTS mínimos sobre 7 tablas por-tenant.
- `withTenant()` (`backend/config/database.config.js`) ahora usa este rol nuevo vía un pool dedicado, con fallback advertido al pool principal si `DATABASE_URL_TENANT_SCOPED` no está configurada.
- Archivos refactorizados de `supabaseAdmin` / `service_role` a `withTenant()` : `AuditorForenseService.js` , `CopilotoService.js` , `EstresadoFinancieroService.js` , `ExtractorService.js` , `ValorExponencialService.js` , `anexos.routes.js` , `proyectos.routes.js` , `matrizRaci.routes.js` , `raciService.js` .
- Prueba de aislamiento cross-tenant real (no supuesta): 5/5 PASS — tenant A no ve filas de tenant B; INSERT con `org_id` ajeno rechazado por policy.
- Tests de regresión: `test:smoke` 8/8 PASS, `test:security` 5/5 PASS.

Deuda conocida, no cerrada hoy: ~264 usos de `getRow/getRows/runSql` en `server.js` y ~130 en el resto de `backend/routes/*.js` siguen sin pasar por `withTenant()` — el pool principal (`postgres` , `BYPASSRLS=true`) los sirve sin backstop de RLS. La única protección ahí es el filtro manual `WHERE org_id = ? / checkOwnership()` en cada ruta, verificado presente en las rutas muestreadas. Migrar los ~400 sitios restantes es un proyecto de refactor de mayor escala, deliberadamente no ejecutado en una sola pasada no supervisada por el riesgo de romper producción sin cobertura de tests suficiente (13 tests totales hoy para 400 call sites).

4. 5 tablas sin RLS y con permisos públicos — vector de ataque cerrado

Durante el trabajo del punto 3 se detectó que 5 tablas nunca tuvieron RLS activado: `gemini_key_state` , `trial_sessions` , `raci_asignaciones` , `raci_roles` , `raci_tareas` (`SELECT relrowsecurity FROM pg_class = false` en las 5).

Hallazgo agravante, confirmado en vivo hoy: los roles `anon` y `authenticated` de Supabase (claves que viven en el bundle del frontend, obtenibles por cualquiera) tenían privilegios **completos** (`SELECT` , `INSERT` , `UPDATE` , `DELETE` , `TRUNCATE` , `REFERENCES` , `TRIGGER`) sobre las 5 tablas vía PostgREST directo — es decir,

cualquiera con la clave pública del proyecto podía leer, modificar o borrar la matriz RACI, las sesiones de prueba y el estado de las llaves Gemini de **cualquier tenant**, sin pasar por el backend ni autenticarse contra él.

Mitigación aplicada y verificada hoy:

```
REVOKE ALL ON gemini_key_state, trial_sessions, raci_asignaciones, raci_rols, raci_tareas
FROM anon, authenticated;
```

Verificación post-cambio: `information_schema.role_table_grants` para `anon / authenticated` sobre las 5 tablas → **0 filas** (sin privilegios remanentes). Confirmado previamente que el frontend no accede a estas tablas vía cliente Supabase directo (`grep .from('raci_tareas'|...)` en `client/src` → 0 resultados) — el REVOKE no rompe ninguna funcionalidad legítima.

Lo que este REVOKE cierra: el vector de ataque externo, no autenticado, vía PostgREST directo. Es la mitigación más urgente y ya está en producción.

Lo que NO cierra (pendiente, distinto del punto anterior): el aislamiento *entre tenants autenticados* dentro del sistema. Estas 5 tablas siguen sin `ENABLE ROW LEVEL SECURITY` ni políticas de tenant — el refactor de código para `matrizRaci.routes.js / raciService.js` a `withTenant()` sí se completó (verificado: diff coherente, sintaxis válida), pero el de `gemini_key_state / trial_sessions` no se alcanzó a iniciar, y la migración SQL que activaría RLS + políticas sobre las 5 tablas (`054_rls_raci_gemini_trial.sql` , referenciada en comentarios del código) **nunca se creó** — el trabajo se cortó por un límite de sesión (rate limit) del agente a cargo, a mitad de la fase de refactor de código, antes de llegar a la fase de base de datos.

5. Incidentes operativos durante el trabajo de hoy

- Un agente ejecutó `taskkill /F /IM node.exe` durante limpieza de un servidor de prueba, matando todos los procesos Node del sistema, no solo el suyo — corregido para la siguiente ejecución (kill por PID específico). Si el usuario tenía otro proceso Node corriendo en ese momento, requirió reinicio manual.
- El trabajo de refactor de las 5 tablas se interrumpió por un límite de sesión de la cuenta (HTTP 429, reset 9:50pm America/Bogota) — no fue un fallo de lógica. El estado del código quedado a medias se auditó manualmente (chequeo de sintaxis en los 11 archivos tocados, `node --check` limpio en todos; diff de los archivos RACI revisado línea por línea, coherente) antes de continuar.

6. Próximos pasos recomendados (no ejecutados hoy)

1. Crear la migración `054_rls_raci_gemini_trial.sql` : `ENABLE ROW LEVEL SECURITY` + políticas de tenant para las 5 tablas — atención especial a `gemini_key_state / trial_sessions` , que probablemente se aíslan por `user_id` , no por `org_id` como las 7 tablas ya migradas (verificar antes de escribir la política, no asumir el mismo patrón).
2. Completar el refactor de código de `gemini_key_state / trial_sessions` a `withTenant()` antes de activar RLS sobre ellas (si no, esas rutas empezarán a devolver 0 filas).
3. Auditar si existen otras tablas con el mismo patrón de `anon / authenticated` con privilegios excesivos — la auditoría de hoy solo cubrió las 5 tablas ya señaladas por el trabajo de RLS, no un barrido completo de todo el esquema.

4. Migrar por fases verificadas los ~400 call sites del pool principal a `withTenant()` (fuera del alcance de una sola sesión).
 5. Verificación manual del usuario en el dashboard de Render: confirmar qué variables de `render.yaml` ya tienen valor real cargado (Stripe, Wompi, Supabase, Sentry, PostHog).
-

Generado el 2026-09-04. Evidencia verificada en vivo contra la base de datos de producción compartida (`ozivmsvxbdtjzkzleqbcy`). No se afirma nada en este documento sin comando ejecutado o archivo: línea que lo respalde.